

Deep Learning for Economics and Finance

Finite-difference and PINN solvers for the HJB+KFE system

Simon Scheidegger

HEC, University of Lausanne

Based on: Achdou, Han, Lasry, Lions & Moll (2022), *Rev. Econ. Stud.* 89(1), 45–86.

Gu, Lauriere, Merkel & Payne (2024), “Global Solutions to Master Equations.” arXiv:2406.13726.

Self-contained slides; Python implementations are introduced after the theory.

Materials: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Recap and Goals

From the Lecture 12 theory deck: We derived the **theory** of CT heterogeneous agent models:

- ▶ Stochastic calculus: Brownian motion, Itô's lemma, Poisson jumps
- ▶ KFE (Fokker–Planck): density evolution, from physics to economics
- ▶ HJB: optimal consumption-savings policy
- ▶ Equilibrium: coupled HJB + KFE + market clearing (Huggett, Aiyagari, KS)
- ▶ Master equation: single PDE encoding the full equilibrium

This deck: How to **solve** these models numerically.

1. **Part I:** Traditional approach (finite differences) — brief overview, limitations
2. **Part II:** **PINNs for the coupled HJB + KFE system** (stationary: Huggett/Aiyagari)
3. **Part III:** **EMINNs for the master equation** (with aggregate shocks: Krusell–Smith)
4. **Part IV:** Results, comparison, and the “deep learning conundrum”

Table of Contents

Traditional Approach: Finite Differences

PINNs for HJB + KFE

EMINNs for the Master Equation

Results, Comparison, and Outlook

Learning objectives

By the end of this lecture you should be able to:

- ▶ Implement finite-difference schemes (upwind discretization) to solve HJB equations on wealth grids
- ▶ Compare finite-difference solutions with PINN and EMINN approaches, recognizing computational trade-offs (speed, accuracy, dimensionality)
- ▶ Deploy physics-informed neural networks to solve the coupled HJB+KFE system for stationary equilibrium in heterogeneous-agent economies
- ▶ Apply equation-constrained multi-output neural networks (EMINNs) to solve the master equation with aggregate shocks (Krusell–Smith)
- ▶ Validate deep-learning solutions against finite-difference benchmarks and assess the deep-learning conundrum in macro-finance

Part I

Traditional Approach: Finite Differences

The Benchmark We Want to Beat

Finite Differences: The Classical Approach

How have economists traditionally solved these models?

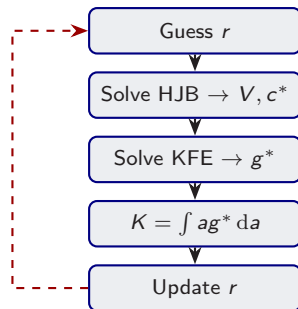
Finite difference method (Achdou et al., 2022):

1. Discretize wealth a on a grid $\{a_1, \dots, a_I\}$
2. Approximate derivatives by finite differences:

$$V'(a_i) \approx \frac{V_{i+1} - V_i}{\Delta a} \quad \text{or} \quad \frac{V_i - V_{i-1}}{\Delta a}$$

3. **Upwind scheme**: use forward diff. when savings > 0 , backward when < 0
4. Iterate HJB $\rightarrow$ policy $\rightarrow$ KFE $\rightarrow$ aggregates $\rightarrow$ prices $\rightarrow$ HJB

This summarizes the finite-difference workflow; code examples are provided later in Python.



Finite Differences: What They Produce

Finite-Difference Example: Aiyagari GE

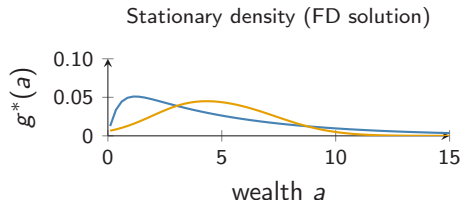
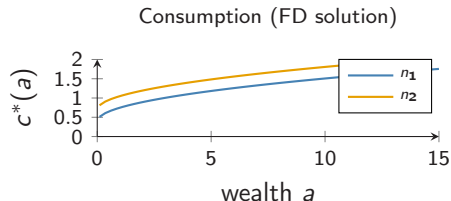
The classical FD approach solves:

- ▶ HJB block: upwind discretization + implicit updates
- ▶ KFE block: transition operator + stationary distribution
- ▶ Price block: root-finding for market-clearing r

Produces:

- ▶ Value function $V(a, n)$
- ▶ Consumption/savings policy $c^*(a, n)$
- ▶ Stationary density $g^*(a, n)$
- ▶ Equilibrium prices r^*, w^*

This is our ground truth for validating deep learning methods.



Why Finite Differences Are Not Enough

FD works well for:

- ✓ Stationary HJB–KFE systems
- ✓ Low-dimensional states (a, n)
- ✓ Monotone convergence theory
- ✓ Fast sparse linear algebra

FD is strained by:

- ✗ Global KS master equation
- ✗ State (a, n, z, g) with distribution g
- ✗ Dense tensor-product grids
- ✗ Moment truncation of g

Takeaway

Neural methods do not remove the equilibrium conditions; they approximate value and distribution maps without building a full grid over g .

Part II

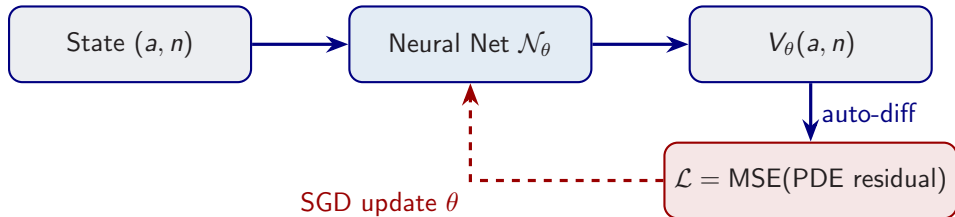
PINNs for the Coupled HJB + KFE System

Solving Huggett and Aiyagari with Neural Networks

Recap: What Is a PINN?

From Block 1 (earlier today): A **Physics-Informed Neural Network** (PINN) solves a PDE by:

1. Parameterizing the solution by a neural network: $u(x) \approx \mathcal{N}_\theta(x)$
2. Computing PDE **residuals** via **automatic differentiation**
3. Minimizing the residual over **collocation points** (no grid needed!)



Key advantage: **mesh-free** — no grid needed, scales to high dimensions.

From Single PDE to Coupled System

Block 1 (PINNs): Solve a *single* PDE (Poisson, HJB cake-eating, Black–Scholes).

Block 2 (this block): Solve a *coupled system* of PDEs arising from equilibrium:

Component	PDE	Unknown
HJB (backward)	$\rho V = \max_c \{u(c) + V' \cdot s + \lambda(\Delta V)\}$	Value $V(a, n)$
KFE (forward)	$0 = -\partial_a[s \cdot g] - \lambda g + \lambda \hat{g}$	Density $g(a, n)$
Market clearing	$K = \sum_n \int a g \, da$	Prices r, w

The coupling: HJB output ($c^* \rightarrow s$) is the KFE input (drift).

KFE output ($g \rightarrow K, L$) determines prices (r, w) that enter HJB.

PINN approach: Parameterize *both* V and g by neural networks.
Minimize *all* residuals **jointly** via a single loss function.

Formal Target System (Stationary, Truncated Domain)

Train on $a \in [\underline{a}, \bar{a}]$, $n \in \{n_1, n_2\}$.

$$\rho V = \max_c \left\{ u(c) + V_a(wn + ra - c) + \lambda(n)(V(a, \hat{n}) - V(a, n)) \right\}, \quad c^* = (V_a)^{-1/\gamma}$$

$$0 = -\partial_a[s(a, n)g(a, n)] - \lambda(n)g(a, n) + \lambda(\hat{n})g(a, \hat{n}), \quad s = wn + ra - c^*$$

Boundary/state constraints:

$$V_a(\underline{a}, n) \geq u'(wn + r\underline{a}), \quad s(\underline{a}, n) \geq 0$$

$$J(a, n) := s(a, n)g(a, n), \quad J(\underline{a}, n) = 0$$

Market clearing:

$$\text{Huggett: } \sum_n \int_{\underline{b}}^{\bar{b}} b g(b, n) db = 0$$

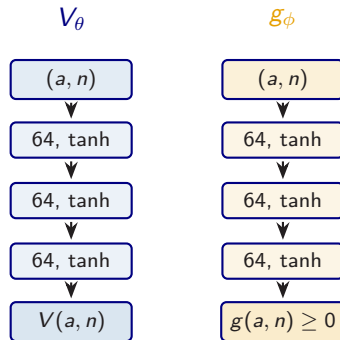
$$\text{Aiyagari: } K^s = \sum_n \int_{\underline{a}}^{\bar{a}} a g(a, n) da = K^d(r)$$

$$\sum_n \int_{\underline{a}}^{\bar{a}} g(a, n) da = 1$$

PINN Architecture for Aiyagari/Huggett

Two neural networks:

- ▶ V_θ : value function network
 - ▶ Input: a (scalar)
 - ▶ Output: 2-vector $(V(a, n_1), V(a, n_2))$, one per income state
 - ▶ Architecture: 4 hidden layers, 64 neurons, tanh
- ▶ g_ϕ : density network
 - ▶ Input: a (scalar)
 - ▶ Output: 2-vector $(g(a, n_1), g(a, n_2)) \geq 0$ (softplus)
 - ▶ Same architecture



Derived quantities (via auto-diff):

- ▶ $V'_\theta = \partial_a V_\theta \Rightarrow$ consumption: $c^* = (V'_\theta)^{-1/\gamma}$
- ▶ $\partial_2 g_\phi \Rightarrow$ KFE residual

Explicit-Policy HJB PINN: $\mathcal{N}_\theta(a, n) \rightarrow (V, c)$

An alternative to recovering c from derivatives of V is to make the policy an explicit network output and let an FOC residual tie the two together.

$$\mathcal{N}_\theta(a, n) = (V_\theta(a, n), c_\theta(a, n)), \quad c_\theta = \text{softplus}(\cdot) + \varepsilon > 0.$$

The loss enforces **both** the HJB residual **and** the FOC consistency:

$$\mathcal{L}^{\text{HJB}} = \underbrace{\frac{1}{M} \sum_m R_{\text{HJB}}(a_m, n_m; V_\theta, c_\theta)^2}_{\text{interior PDE residual}} + \kappa_{\text{FOC}} \underbrace{\frac{1}{M} \sum_m (c_\theta(a_m, n_m)^{-\gamma} - V'_{\theta,a}(a_m, n_m))^2}_{\text{first-order condition}}.$$

Why this is more teachable than $c = (V_a)^{-1/\gamma}$:

- ▶ The FOC is an **explicit residual** students can monitor.
- ▶ No early-training pathologies from $V_a \leq 0$.
- ▶ Policy is a smooth network output, plots stable from epoch 1.
- ▶ FOC residual must vanish *independently* of the HJB residual.

It generalises to two discrete income states and to OU diffusion income; Notebook 08 instead takes the $c = (V_a)^{-1/\gamma}$ route with log utility ($\gamma = 1$, so $c = 1/V_a$) and outputs (V, g) for the coupled HJB-KFE system.

PINN Objective: Interior + Boundary + Equilibrium

The total loss combines **all equilibrium conditions**:

$$\mathcal{L} = \kappa_{\text{HJB}} \mathcal{L}_{\text{HJB}}^{\text{int}} + \kappa_{\text{KFE}} \mathcal{L}_{\text{KFE}}^{\text{int}} + \kappa_{\text{KFE,cum}} \mathcal{L}_{\text{KFE}}^{\text{cum}} + \kappa_{\text{SC}} \mathcal{L}_{\text{SC}} + \kappa_{\text{flux}} \mathcal{L}_{\text{flux}} + \kappa_{\text{mass}} \mathcal{L}_{\text{mass}} + \kappa_{\text{MC}} \mathcal{L}_{\text{MC}}$$

Component	Definition	What it enforces
$\mathcal{L}_{\text{HJB}}^{\text{int}}$	$\frac{1}{M} \sum_m R_{\text{HJB}}(a_m, n_m) ^2$	Interior HJB
$\mathcal{L}_{\text{KFE}}^{\text{int}}$	$\frac{1}{M} \sum_m R_{\text{KFE}}(a_m, n_m) ^2$	Interior KFE
$\mathcal{L}_{\text{KFE}}^{\text{cum}}$	$\frac{1}{I} \sum_k R_{\text{KFE}}^{\text{cum}}(a_k) ^2$	Integrated KFE balance
$\mathcal{L}_{\text{SC}}$	$\sum_n \max(0, -s_\theta(\underline{a}, n))^2$	State constraint
$\mathcal{L}_{\text{flux}}$	$\sum_n (J(\underline{a}, n)^2 + J(\bar{a}, n)^2)$	Boundary fluxes
$\mathcal{L}_{\text{mass}}$	$(\sum_n \int g_\phi \, da - 1)^2$	Normalization
$\mathcal{L}_{\text{MC}}$	Huggett: $(\int bg)^2$; Aiyagari: $(K^s - K^d(r))^2$	Market clearing

Softplus output enforces $g_\phi \geq 0$. Weights can be adapted with ReLoBRaLo.

HJB Residual: Computed via Auto-Differentiation

The interior HJB residual at collocation point (a, n) . In notebook 08, consumption is recovered from the value derivative, $c^* = (V'_\theta)^{-1/\gamma}$. In the explicit-policy variant of the previous slide, replace c^* by the network output c_θ and add an FOC residual.

$$R_{\text{HJB}}(a, n) = \rho V_\theta - u(c^*) - V'_\theta \cdot (wn + ra - c^*) - \lambda(n)(V_\theta(a, \hat{n}) - V_\theta(a, n))$$

```
def hjb_residual(model_V, a, n, r, w, params):
    a.requires_grad_(True)
    V = model_V(a, n)
    V_a = torch.autograd.grad(V, a, ...)[0] # auto-diff
    c_star = V_a.clamp(min=1e-8) ** (-1.0 / gamma) # FOC
    s = w * n + r * a - c_star # savings
    V_hat = model_V(a, n_hat) # value in other state
    res = rho * V - u(c_star) - V_a * s
        - lam(n) * (V_hat - V)
    return res
```


Integrated KFE: A Stable Companion

Differentiating a noisy density g_ϕ pointwise is fragile; cumulative probability balance is often more stable.

Local (strong-form) residual.

$$R_{\text{KFE}}(a, n) = \partial_a[s g_\phi] + \lambda(n)g_\phi(a, n) - \lambda(\hat{n})g_\phi(a, \hat{n}).$$

Wiggles in g_ϕ produce large ∂_a noise.

Integrated flux balance on a deterministic grid $\{a_k\}$, with $J_j = s_j g_j$:

$$R_{1,k}^{\text{cum}} = J_1(a_k) - J_1(\underline{a}) + \int_{\underline{a}}^{a_k} [\lambda_{12}g_1(u) - \lambda_{21}g_2(u)] \, du,$$

$$R_{2,k}^{\text{cum}} = J_2(a_k) - J_2(\underline{a}) + \int_{\underline{a}}^{a_k} [\lambda_{21}g_2(u) - \lambda_{12}g_1(u)] \, du.$$

No $\partial_a g_\phi$; only quadrature.

- ▶ Cheap: one quadrature per epoch on a fixed grid.
- ▶ Improves stationary-density diagnostics in notebook 08.
- ▶ Use both: $\mathcal{L}^{\text{KFE}} = \kappa_{\text{loc}}\mathcal{L}_{\text{loc}} + \kappa_{\text{cum}}\mathcal{L}_{\text{cum}}$.

KFE Residual: Density Must Satisfy the Forward Equation

The interior stationary KFE residual at (a, n) :

$$R_{\text{KFE}}(a, n) = \partial_a[s(a, n) g_\phi(a, n)] + \lambda(n) g_\phi(a, n) - \lambda(\hat{n}) g_\phi(a, \hat{n})$$

```
def kfe_residual(model_g, model_V, a, n, r, w, params):
    a.requires_grad_(True)
    g = model_g(a, n)
    c_star = get_consumption(model_V, a, n, params)
    s = w * n + r * a - c_star
    flux = s * g  # flux = savings * density
    d_flux = torch.autograd.grad(flux, a, ...)[0]
    g_hat = model_g(a, n_hat)  # density in other state
    res = d_flux + lam(n) * g - lam(n_hat) * g_hat
    return res
```

Boundary, Flux, and Market-Clearing Terms

Aiyagari market clearing:

$$\mathcal{L}_{\text{MC}} = \left(\sum_n \int_{\underline{a}}^{\bar{a}} a g_{\phi} da - K^d(r) \right)^2$$

For Huggett replace with:

$$\mathcal{L}_{\text{MC}}^{\text{H}} = \left(\sum_n \int_{\underline{b}}^{\bar{b}} b g_{\phi}(b, n) db \right)^2.$$

Normalization: $\mathcal{L}_{\text{mass}} = \left(\sum_n \int_{\underline{a}}^{\bar{a}} g_{\phi}(a, n) da - 1 \right)^2$

State-constraint penalty:

$$\mathcal{L}_{\text{SC}} = \sum_n \max(0, -s_{\theta}(\underline{a}, n))^2$$

Boundary flux penalty (truncated domain $[\underline{a}, \bar{a}]$):

$$\mathcal{L}_{\text{flux}} = \sum_n \left(J_{\phi}(\underline{a}, n)^2 + J_{\phi}(\bar{a}, n)^2 \right), \quad J_{\phi}(a, n) = s_{\theta}(a, n) g_{\phi}(a, n)$$

Integrals are evaluated via Monte Carlo or quadrature.

Training Algorithm: Jointly Solving HJB + KFE

PINN Training at FD-Computed Equilibrium Prices (Notebook 08)

- 1: Initialize V_θ , g_ϕ ; fix prices at the FD equilibrium (r^*, w^*)
- 2: **for** $k = 1, \dots, N_{\text{epochs}}$ **do**
- 3: Sample M collocation points $\{(a_m, n_m)\}$
- 4: Compute $c_m^* = (V'_\theta(a_m))^{-1/\gamma}$
- 5: Compute R_{HJB}^m , R_{KFE}^m via auto-diff
- 6: Integrate KFE on a fixed asset grid
- 7: Compute $\mathcal{L}_{\text{SC}}, \mathcal{L}_{\text{flux}}, \mathcal{L}_{\text{mass}}, \mathcal{L}_{\text{MC}}$ ($\mathcal{L}_{\text{MC}}$ at fixed r^*)
- 8: Total loss: $\mathcal{L} = \sum \kappa_i \mathcal{L}_i$; update θ, ϕ (Adam)
- 9: **end for**
- 10: Freeze collocation grid; run FP64 L-BFGS polish

Key choices:

- **Prices:** fixed at the FD (r^*, w^*) ; outer $r \leftarrow r + \alpha(K^s - K^d)$ updates $\Rightarrow$ GE PINN
- **Collocation:** sample (a, n) uniformly on $[\underline{a}, \bar{a}] \times \{n_1, n_2\}$
- **Loss weights:** start with $\kappa_{\text{HJB}} = \kappa_{\text{KFE}} = 1$, larger $\kappa_{\text{SC}}, \kappa_{\text{flux}}, \kappa_{\text{MC}}$
- **Optimizer:** Adam, lr $\sim 10^{-3}$, then FP64 L-BFGS
- **Validation:** compare against FD only after training

Same philosophy as the PINN lecture:
economic equations = loss function; auto-diff =
all derivatives.

Applying the PINN: Huggett (1993) Model

Huggett model (endowment economy, zero net supply of bonds):

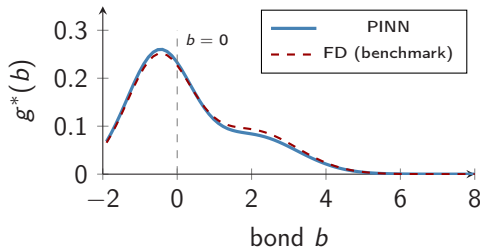
Differences from Aiyagari:

- ▶ Asset: risk-free bond b (not capital)
- ▶ Budget: $db = (qb + n - c) dt$
- ▶ Market clearing: $\sum_n \int b g(b, n) db = 0$
- ▶ Price: bond return q (not interest rate via firm FOC)

PINN modification:

- ▶ Same architecture (V_θ, g_ϕ)
- ▶ HJB: replace ra by qb , remove wage
- ▶ $\mathcal{L}_{MC} = \left(\int b g_\phi db \right)^2$
- ▶ Iterate on q instead of r

Huggett: PINN vs. FD



PINN closely matches the FD benchmark.

Applying the PINN: Aiyagari (1994) Model

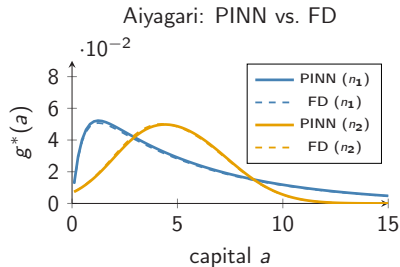
Aiyagari model (production economy, capital market clearing):

vs. Huggett: two prices (r, w) from firm FOC,
 $\int ag = K, \int ng = L, r = \alpha K^{\alpha-1} L^{1-\alpha} - \delta.$

PINN diagnostics (notebook 08, longer in-class run):

Diagnostic	Value
$K^{\text{FD}} / K^{\text{PINN}}$	0.30893 / 0.30894
$ K^{\text{FD}} - K^{\text{PINN}} $	8.7×10^{-6}
c MSE	4.7×10^{-4}
g MSE	8.1×10^{-4}
KS dist. z_1/z_2	0.066/0.059
$\max R_{\text{HJB}} $	5.7×10^{-2}
$\max R_{\text{KFE}} $	2.7×10^{-1}

Honest read: K matches to 5 digits; pointwise density and KS gaps are non-trivial. The integrated-KFE term targets exactly the loosest residual. The checked-in smoke run is shorter and prints larger gaps.



Right-skewed, mass at $\underline{a}$: reproduced by PINN.

Handling the Borrowing Constraint

The borrowing constraint $a \geq \underline{a}$ is a key challenge for PINNs.

Option A: Soft constraint (penalty)

Add penalty: $\psi(a) = -\frac{1}{2}\kappa(a - \underline{a})^2$ for $a \leq \underline{a}$

Advantages:

- ▶ Simple to implement
- ▶ Smooth loss landscape
- ▶ Avoids mass point issues in KFE

Option B: Hard constraint (trial solution)

Architecture: $V_\theta(a, n) = (a - \underline{a})\hat{V}_\theta + V_{\text{BC}}$

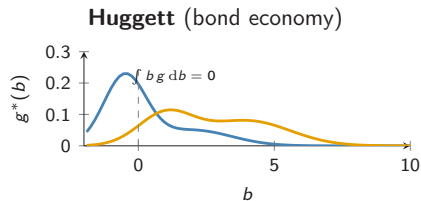
Advantages:

- ▶ Exact BC satisfaction
- ▶ No tuning of κ
- ▶ Recall Chapter 7 (PINNs), Notebook 02

Recommendation (Achdou et al., 2022; Gu et al., 2024):

Use the **soft borrowing constraint** $\psi(a)$. The inequality BC causes technical problems (non-differentiable V , mass point in g). Softening avoids these while being economically faithful for κ large enough.

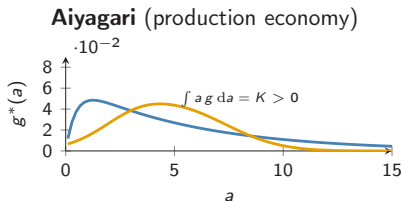
PINN Solutions: Comparing Huggett and Aiyagari Distributions



- ▶ Mean wealth = 0 (zero net supply)
- ▶ Large mass at $\underline{b}$ (constrained)
- ▶ $q^* < \rho$ (low return)

The PINN correctly captures these qualitative differences. Same code architecture, different market clearing conditions.

Schematic. Curves are illustrative; actual PINN outputs from NB 08 are slightly noisier near the borrowing limit and in the right tail.



- ▶ Mean wealth = $K > 0$ (capital stock)
- ▶ Long right tail (wealthy agents)
- ▶ $r^* < \rho$ but capital earns return

Why PINNs over Finite Differences?

For stationary problems (Huggett/Aiyagari without aggregate shocks):

	Finite Differences	PINN
Grid required	Yes (I points)	No (collocation)
Upwind scheme	Manual implementation	Auto-diff handles it
Curse of dimensionality	$O(I^d)$	$O(M \cdot \theta)$
Extension to higher dim	Very hard	Natural
Borrowing constraint	Special treatment	Soft penalty / hard BC
Accuracy (2D)	$\sim 10^{-6}$	$\sim 10^{-4}$
Speed (2D)	Faster	Slower

Verdict: *FD is faster and more accurate* for low-dim stationary problems — use it as the gold-standard benchmark. PINNs **scale to high dimensions**: aggregate shocks (KS, state $\ni g$), multiple assets, richer heterogeneity.

$\Rightarrow$ Part III: **EMINNs** for the master equation.

Part III

EMINNs: Solving the Master Equation

Economic Model Informed Neural Networks

Gu, Lauriere, Merkel & Payne (2024)

Note: the companion EMINN notebook is deferred to the next iteration of the course; this Part III is conceptual.

The Challenge: Aggregate Shocks

Krusell–Smith (1998): Add aggregate TFP shocks $dZ_t = \eta(\bar{Z} - Z_t) dt + \sigma^z dB_t^0$.

Now the value function depends on the **entire distribution**:

$$V = V(a, n, z, g) \quad \text{where } g \text{ is infinite-dimensional}$$

Why is this hard?

- ▶ Cannot discretize g on a grid (infinite-dimensional)
- ▶ Original KS trick: $g \approx K$ (first moment). But loses distributional information.
- ▶ We want **global** solutions: accurate for all (z, g) , not just near steady state

The Master Equation (Lecture 12)

Substitute KFE + market clearing + belief consistency into HJB $\Rightarrow$ **single PDE**:

$$\mathcal{L}V(a, n, z, g) = 0$$

One PDE to solve, but with an infinite-dimensional argument g .

Approximating the Distribution: Three Approaches

Key idea: Replace the infinite-dimensional g by a **finite-dimensional** approximation $\hat{\varphi}$.

	Finite Population	Discrete State	Projection
Dist. params $\hat{\varphi}_t$	$\{(a_t^i, n_t^i)\}_{i \leq N}$	Masses on grid	Basis coefficients
Dist. approx. $\hat{g}_t$	$\frac{1}{N} \sum \delta_{\hat{\varphi}_t^i}$	$\sum \hat{\varphi}_{i,j,t} \delta_{(a_i, n_j)}$	$b_0 + \sum \hat{\varphi}_{i,t} b_i(a, n)$
KFE approx.	Other agents' evolution	FD mass evolution	Coeff. evolution
Dimension	~ 40	~ 200	~ 5
$K_t =$	$\frac{1}{N} \sum a_t^i$	$\sum_j a_j \hat{\varphi}_{j,t}$	$\int a \sum \hat{\varphi}_{i,t} b_i da$

After approximation: $V(a, n, z, g) \approx \hat{V}(a, n, z, \hat{\varphi})$ where $\hat{\varphi} \in \mathbb{R}^N$.

Now the master equation becomes a **finite-dimensional PDE** that a neural network can solve!

Approach 1: Finite Population ($N \approx 40$ agents)

Replace continuum by N agents with states $\hat{\varphi}_t = \{(a_t^1, l_t^1), \dots, (a_t^N, l_t^N)\}$.

Approximate capital: $K_t = \frac{1}{N} \sum_{i=1}^N a_t^i$

Why are 40 agents sufficient?

- ▶ We sample individual states (a, n) and distribution states $(\hat{\varphi})$ **separately** when generating training data
- ▶ For individual policy: can concentrate on high-curvature regions (near $\underline{a}$) independently of N
- ▶ For aggregate capital: LLN kicks in quickly — 40 agents give accurate K
- ▶ Can analytically eliminate finite-agent noise in $\hat{\varphi}$ -drift during training

This is much smaller than discrete-time simulation methods (e.g., Young's method from Chapter 6 used $\sim 10,000$ agents).
Continuous-time formulation helps!

Approach 2: Discrete State Space (~ 200 grid points)

Discretize wealth on a grid $\{a_1, \dots, a_I\}$. Distribution $\approx$ masses on grid:

$$\hat{g}_t(a, n) \approx \sum_{i=1}^I \sum_{j=1}^2 \hat{\varphi}_{i,j,t} \delta_{(a_i, n_j)}$$

KFE becomes: evolution of grid masses via **finite differences**.

Master equation: functional derivative w.r.t. g becomes a **partial derivative** w.r.t. each mass $\hat{\varphi}_{m,j}$:

$$\hat{\mathcal{L}}_g \hat{W} = \sum_{j=1,2} \sum_{m=1}^I \mu_{\hat{\varphi},m,j}(z, \hat{\varphi}) \partial_{\hat{\varphi}_{m,j}} \hat{W}$$

Dimension: $\hat{\varphi} \in \mathbb{R}^{2I}$ with $I \approx 100$, so $\dim(\hat{\varphi}) \approx 200$.

Neural network: needs an **embedding** to handle the 200-dim input efficiently.

$\Rightarrow$ Use a recurrent or set-encoder architecture (Sirignano & Spiliopoulos, 2018).

Approach 3: Projection onto Basis Functions (~ 5 components)

Idea: Project g onto a low-dimensional basis: $\hat{g}_t(a, n) = b_0(a, n) + \sum_{i=1}^N \hat{\varphi}_{i,t} b_i(a, n)$

Choice of basis (Gu et al., 2024):

- ▶ Use **eigenfunctions** of the steady-state KFE operator $\bar{\mathcal{L}}^{KF}$
- ▶ These are the **most persistent** components of the density
- ▶ Intuition: most persistent components carry most price-relevant information
- ▶ Only $N \approx 5$ basis functions needed!

Dimension: $\hat{\varphi} \in \mathbb{R}^5$. Total NN input: $(a, n, z, \hat{\varphi}) \in \mathbb{R}^{2+1+5} = \mathbb{R}^8$.

Projection is **lowest-dimensional** but requires the most intricate setup (computing eigenfunctions, choosing test functions for KFE evolution).

Comparing the Three Approximation Approaches

	Finite Population	Discrete State	Projection
Dimension of $\hat{\varphi}$	~ 40	~ 200	~ 5
NN architecture	Feed-forward	Recurrent + embedding	Feed-forward
Initialization	$W(a, \cdot) = e^{-a}$	Random	Random
Customization	Choose N	Grid, FD method	Basis, test functions
Conceptual difficulty	Low	Medium	High

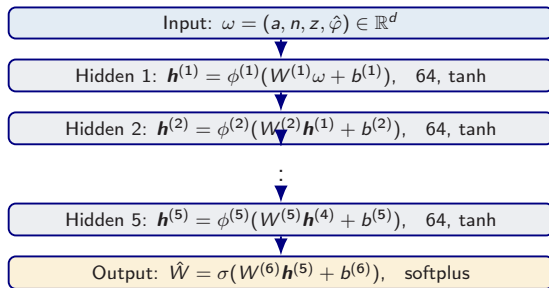
Trade-off:

- ▶ Finite population: simple concept, moderate dimension, works well in practice
- ▶ Discrete state: most similar to FD, highest dimension, needs embedding
- ▶ Projection: **lowest dimension**, but requires careful choice of basis and test functions

All three approaches are validated in Gu et al. (2024). They produce **consistent results**.

Neural Network for $W := \partial_a V$

Parameterize $W(\omega) \approx \hat{W}_\Theta(\omega)$ where $\omega = (a, n, z, \hat{\varphi})$:



- ▶ $H = 5$ hidden layers, 64 neurons each, tanh activation
- ▶ Output: **softplus** $\sigma(x) = \log(1 + e^x)$ ensures $W > 0$ (marginal value positive)
- ▶ Parameters: $\Theta = (W^{(1)}, b^{(1)}, \dots, W^{(6)}, b^{(6)})$
- ▶ Optimal consumption: $c^* = W^{-1/\gamma}$ (directly from the envelope condition)

The EMINN Algorithm

Economic Model Informed Neural Network (EMINN)

- 1: **Initialize** NN $\hat{W}_\Theta$ with parameters Θ
- 2: **while** Loss > tolerance **do**
- 3: **Sample** M points: $\mathbf{S} = (a_m, n_m, z_m, \hat{\varphi}_m)_{m=1}^M$
- 4: **Compute** loss: $\mathcal{E} = \kappa^e \frac{1}{M} \sum_m |\hat{\mathcal{L}}(\cdot; \Theta)|^2 + \kappa^s \mathcal{E}^s(\mathbf{S}; \Theta)$
- 5: $\hat{\mathcal{L}}$: master eq. residual (automatic differentiation); $\mathcal{E}^s$: shape penalty
- 6: **Update** $\Theta \leftarrow \Theta - \alpha \nabla_\Theta \mathcal{E}$
- 7: **end while**

This is exactly a PINN applied to the master equation!

- ▶ “Physics” = economic equilibrium conditions (master equation)
- ▶ All derivatives (including w.r.t. $\hat{\varphi}$) computed by **automatic differentiation**

Computing the Master Equation Residual

The residual $\hat{\mathcal{L}}$ decomposes into three operators:

$$0 = (r(z, \hat{\varphi}) - \rho) \hat{W} + \underbrace{\hat{\mathcal{L}}_x \hat{W}}_{\text{individual}} + \underbrace{\hat{\mathcal{L}}_z \hat{W}}_{\text{TFP}} + \underbrace{\hat{\mathcal{L}}_g \hat{W}}_{\text{distribution}}$$

Operator	Expression
$\hat{\mathcal{L}}_x \hat{W}$	$s(\cdot) \partial_a \hat{W} + \lambda(n) (\hat{W}(\hat{n}) - \hat{W}(n))$ (savings + income switch)
$\hat{\mathcal{L}}_z \hat{W}$	$\partial_z \hat{W} \cdot \mu^z(z) + \frac{1}{2} (\sigma^z)^2 \partial_{zz} \hat{W}$ (OU aggregate shock)
$\hat{\mathcal{L}}_g \hat{W}$	$\sum_n \frac{\partial \hat{W}}{\partial \hat{\varphi}_n} \cdot \mu_{\hat{\varphi}, n}(z, \hat{\varphi})$ (distribution evolution)

All derivatives ($\partial_a, \partial_z, \partial_{zz}, \partial_{\hat{\varphi}_n}$) computed by **one backward pass** through the neural network. This is the power of automatic differentiation: even the functional derivative (w.r.t. distribution) is just a partial derivative in the finite-dimensional approximation.

Shape Constraints: Avoiding “Cheat Solutions”

The Macroeconomic Deep Learning Conundrum

The algorithm is straightforward to describe and code... but **hard to implement successfully!** NNs may get stuck in bad solutions (constant, non-increasing, non-concave V).

Solution: Add **shape penalties** $\mathcal{E}^s$ to the loss:

- ▶ **Concavity:** penalize $\partial_{aa} V > 0$: $\mathcal{E}^{\text{concav}} = \frac{1}{M} \sum \max(0, \partial_{aa} V_m)^2$
- ▶ **Monotonicity:** penalize $\partial_{za} V > 0$ (higher $z \Rightarrow$ lower marginal value)

Additional tricks:

- ▶ **Initialization:** $W(a, \cdot) = e^{-a}$ (correct qualitative shape from the start)
- ▶ **Architecture:** train $\phi(a, n, z, g; \theta) \cdot (a - \underline{a})^{1-\gamma}$ (correct asymptotics)
- ▶ **Active sampling:** sample more points where the residual is large

Sampling Strategies: Where to Evaluate the Residual

Challenge: state space $(a, n, z, \hat{\varphi})$ is high-dimensional. Must sample from **economically relevant** regions.

Distribution $\hat{\varphi}$ sampling:

1. **Moment:** draw K , then $\hat{\varphi}$ consistent with K
2. **Mixed steady-state:** solve SS for various z , take mixtures
3. **Ergodic:** simulate economy with current $\hat{W}$, use simulated distributions

Individual states (a, n) :

- ▶ Uniform on $[\underline{a}, \bar{a}] \times \{l_1, l_2\}$
- ▶ Add draws where error is high (**active sampling**)

Aggregate state z : Uniform on $[z_{\min}, z_{\max}]$

Separation of individual and distribution sampling is key to efficiency.

Implementation Details

	Finite Pop.	Discrete State	Projection
Neural Network			
Structure	Feed-forward	RNN/embed.	Feed-forward
Initialization	$W(a, \cdot) = e^{-a}$	Random	Random
Sampling			
(a, n)	Active	Uniform	Uniform
$\hat{\phi}$	Moment sampling	Mixed steady-state	Ergodic
z	$U[z_{\min}, z_{\max}]$	$U[z_{\min}, z_{\max}]$	$U[z_{\min}, z_{\max}]$
Loss Function			
Shape constraints	$\partial_{aa} V < 0, \partial_{za} V < 0$	Same	$\partial_{aa} V < 0$
Weights	$\kappa^e = 100, \kappa^s = 1$	$\kappa^e = \kappa^s = 1$	$\kappa^e = \kappa^s = 1$

Q&A: Key differences from discrete time?

- ▶ Derivatives replace expectations, so automatic differentiation is essential.
- ▶ Collocation and shape constraints replace simulation grids and false-time steps.

Part IV

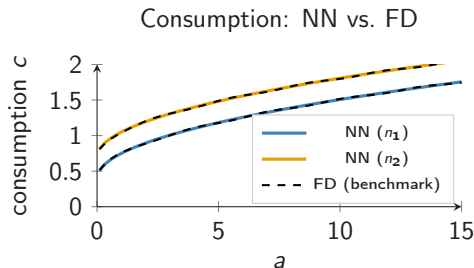
Results, Comparison, and Outlook

Results: Aiyagari Model (No Aggregate Shocks)

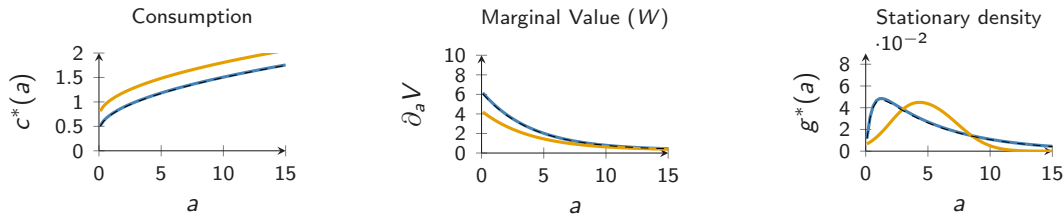
Validation: Compare EMINN solutions against finite-difference benchmark.

Method	ME Loss	MSE(NN,FD)
Finite Pop. NN	3.1×10^{-5}	4.8×10^{-5}
Discrete State NN	9.3×10^{-6}	6.6×10^{-5}

- ▶ ME residual: $\sim 10^{-5}$ (excellent)
- ▶ Agreement with FD: $\sim 10^{-5}$ MSE
- ▶ Both approaches match FD closely
- ▶ Consistent across multiple runs



Aiyagari: Consumption, Marginal Value, and Distribution



Solid curves: NN

Dashed curves: FD benchmark

NN and FD are very close across all three panels.

Schematic. Curves are illustrative; for the actual NB 08 outputs (with PINN jitter visible near a and in the right tail) run the notebook end to end.

Reported Results: Krusell–Smith with Aggregate Shocks

Gu et al. (2024) report global master-equation solutions with aggregate TFP shocks.

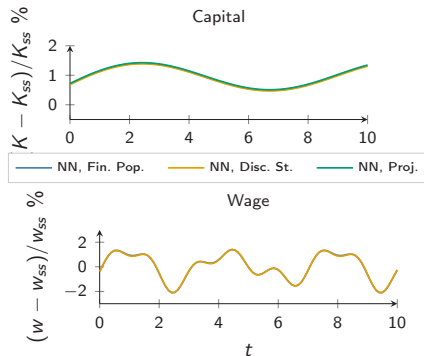
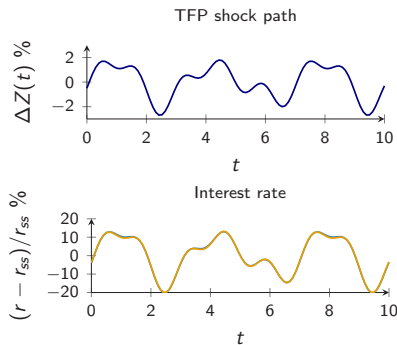
Method	Master Eq. Training Loss
Finite Pop. NN	3.0×10^{-5}
Discrete Grid NN	9.6×10^{-5}
Projection NN	8.5×10^{-6}

All three approaches achieve reported training losses around 10^{-5} .

Comparison with Fernandez-Villaverde, Hurtado & Nuno (2023):

- ▶ Their method: perturbation around steady state
- ▶ EMINN: neural global approximation of the master equation
- ▶ Reported paths match closely for moderate shocks; differences matter farther from steady state

Krusell–Smith: Sample Time Paths



All three NN approaches produce **nearly identical** time paths for aggregate variables.

Schematic. Curves are illustrative impulse responses, not exports from a trained EMINN run; for the actual reported numerics see Gu, Laurière, Merkel & Payne (2024), Fig. 5–7.

Method Comparison: FD vs. PINN vs. EMINN

	Finite Diff.	PINN	EMINN
Global master eq.	✗	✗	✓
Stationary equilibrium	✓	✓	✓
Grid required	Yes	No	No
High-dimensional scaling	Limited	Better, validate	Better, validate
Handles g as state	Standard: no	No	Via $\hat{\phi}$
Theory	Strong in low dim	Limited	Limited
Low-dim accuracy	$\sim 10^{-6}$	problem-dependent	reported $\sim 10^{-5}$

Bottom line:

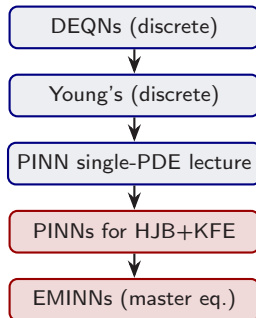
- ▶ **Stationary, low-dim:** FD is fast and reliable. Use it as benchmark.
- ▶ **Aggregate shocks:** EMINNs target global master-equation solutions.
- ▶ **PINNs:** useful bridge for the coupled HJB–KFE system.

The Deep Learning Advantage

Why deep learning for het. agent macro?

1. **Meshless**: no exponential grid growth
2. **Handles ∞ -dim states**: distribution g approximated by finite $\hat{\varphi}$, NN handles the rest
3. **Auto-diff**: all derivatives (incl. functional) computed automatically
4. **Global**: accurate for all states, not just near steady state
5. **Flexible**: same framework for Huggett, Aiyagari, KS, HANK, ...

Connection to this course:



Open Challenges

The Macroeconomic Deep Learning Conundrum (Gu et al., 2024)

“The algorithm is straightforward to describe and code... but hard to implement successfully!”

Key challenges:

1. **Training stability:** NN can get stuck in bad local minima (“cheat solutions”)
 - ▶ Constant, non-increasing, or non-concave $V \Rightarrow$ shape penalties needed
2. **No convergence guarantees:** unlike FD, we cannot prove the NN converges
3. **Sensitivity to hyperparameters:** architecture, lr, sampling, loss weights all matter
4. **Verification:** **must** compare against known solutions (FD, perturbation)
 - ▶ First validate on Aiyagari (no agg. shocks), then extend to KS
5. **Computational cost:** training can take hours/days on GPU for large models

Active research area: better architectures, better sampling, better loss functions.

Extensions and Future Directions

The EMINN framework extends naturally to richer models:

Model extensions:

- ▶ **HANK**: add nominal rigidities (Kaplan, Moll & Violante, 2018)
- ▶ **Multi-asset**: bonds + stocks + housing (higher-dim individual state)
- ▶ **Life-cycle**: age as additional state (more heterogeneity)
- ▶ **Non-convex adjustment costs** (e.g., lumpy investment)

Methodological extensions:

- ▶ **Better sampling**: importance sampling, adaptive collocation
- ▶ **Transfer learning**: pre-train on simpler model, fine-tune on complex one
- ▶ **Multi-scale training**: curriculum from easy → hard
- ▶ **Hybrid FD-NN**: use FD for low-dim parts, NN for high-dim parts

The frontier: Can we solve HANK models with aggregate shocks **globally**?
This is one of the most important open problems in computational macro.

Summary and Key Takeaways

FD (Benchmark)

Fast and accurate for **stationary, low-dim** problems.
Gold standard for Aiyagari/Huggett.

PINN (Coupled PDEs)

Mesh-free solution of HJB + KFE via auto-diff. Same philosophy as Chapter 7.

EMINN (Master Eq.)

Global solutions with aggregate shocks. Handles distribution as state.

The big picture:

1. CT het. agent models = coupled HJB + KFE + market clearing
2. Deep learning (PINNs/EMINNs) embeds equilibrium conditions in the loss
3. Auto-differentiation computes all derivatives (including functional derivatives)
4. For aggregate shocks: approximate g in finite dimensions, solve master equation
5. Validated against FD benchmarks — works, but training requires care

Next chapter: Gaussian Processes (Chapter 9); deep surrogate models follow in Chapter 10.

Code Companion: Run This Live

Companion notebook for this lecture (self-contained, PyTorch):

- ▶ **NB 08** (*in-class*): `lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`
— the stationary Aiyagari equilibrium: an upwind finite-difference solver computes the GE prices, then a PINN re-solves the coupled HJB+KFE system at those prices, with FD as the validation benchmark.

Suggested in-class flow: run the notebook top to bottom — (i) FD dense baseline and the sparse-FD cross-check, (ii) the GE bisection on r , (iii) the PINN re-solve at (r^*, w^*) , (iv) the PINN-vs-FD agreement diagnostics. The single-PDE HJB warm-ups (discrete-income and OU-diffusion) are PINN exercises and live with the PINN lecture (Lecture 11).

References

Core papers:

- ▶ Achdou, Y., Han, J., Lasry, J.-M., Lions, P.-L. & Moll, B. (2022). Income and Wealth Distribution in Macroeconomics: A CT Approach. *REStud* 89(1), 45–86.
- ▶ Gu, Z., Lauriere, M., Merkel, S. & Payne, J. (2024). Global Solutions to Master Equations for CT HA Models. arXiv:2406.13726.
- ▶ Fernandez-Villaverde, J., Hurtado, S. & Nuno, G. (2023). Financial Frictions and the Wealth Distribution. *Econometrica* 91(3), 869–901.

Foundational:

- ▶ Aiyagari, S.R. (1994). Uninsured Idiosyncratic Risk and Aggregate Saving. *QJE* 109(3), 659–684.
- ▶ Huggett, M. (1993). The Risk-Free Rate in HA Economies. *JEDC* 17(5–6), 953–969.
- ▶ Krusell, P. & Smith, A.A. (1998). Income and Wealth Heterogeneity. *JPE* 106(5), 867–896.

Mean field games and DL:

- ▶ Lasry, J.-M. & Lions, P.-L. (2007). Mean Field Games. *Jpn. J. Math.* 2(1), 229–260.
- ▶ Sirignano, J. & Spiliopoulos, K. (2018). DGM: A Deep Learning Algorithm for Solving PDEs. *J. Comput. Phys.* 375, 1339–1364.

HANK and applications:

- ▶ Kaplan, G., Moll, B. & Violante, G.L. (2018). Monetary Policy According to HANK. *AER* 108(3), 697–743.
- ▶ Ahn, S., Kaplan, G., Moll, B., Winberry, T. & Wolf, C. (2018). When Inequality Matters for Macro. *NBER Macro Annual* 32(1), 1–75.
- ▶ Gabaix, X., Lasry, J.-M., Lions, P.-L. & Moll, B. (2016). The Dynamics of Inequality. *Ecta* 84(6), 2071–2111.
- ▶ Nuno, G. & Moll, B. (2018). Social Optima in Economies with HA. *RED* 28, 150–180.

Moll lecture notes:

- ▶ benjaminmoll.com/lectures/ and benjaminmoll.com/codes/

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: [https:](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[//github.com/sischei/Deep_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Learning_for_Solving_And_](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

[Estimating_Dynamic_Economic_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

Covered today (Deck 2):

- I FD: The Benchmark
- II PINNs for HJB+KFE
- III EMINNs for Master Eq.
- IV Results & Outlook

Where to Next:

Surrogate Models
& Gaussian Processes